Product Requirements Document (PRD)

E-commerce Platform with Admin Dashboard

Document Version	1.0
Status	Draft — Pending Stakeholder Approval
Product Type	Single-store E-commerce Web Application
Primary Payment Method	Cash on Delivery (COD)
Primary Language	English for technical documentation; customer-facing language to be confirmed.

1. Product Overview

1.1 Product Vision

Build a web-based e-commerce platform that enables customers to discover products, place orders, and track their purchases, while allowing authorized administrators and staff to manage the store through a centralized dashboard.

The platform will support product and category management, inventory tracking, order processing, customer account management, and role-based access control.

1.2 Problem Statement

Managing products, stock quantities, customer information, and orders manually can result in operational inefficiencies, inconsistent inventory, and difficulty tracking order progress.

The platform aims to centralize store operations, reduce manual work, and provide customers with a straightforward shopping experience.

1.3 Proposed Solution

The platform will consist of two main interfaces:

A. Customer Storefront

- Browse and search products.
- Explore product categories.
- View product details, prices, images, and availability.
- Add products to a shopping cart.
- Submit orders using Cash on Delivery.
- View order information and status.
- Manage customer account information, where applicable.

B. Admin Dashboard

- View store statistics and recent activity.
- Create, edit, archive, and manage products.
- Create and manage product categories.
- Monitor stock quantities.
- View and process customer orders.
- Update order statuses.
- View customer accounts and order histories.
- Manage staff accounts and permissions.
- Review important administrative actions.

1.4 Product Scope

The product is a **single-store platform**, not a multi-vendor marketplace. One business owns the catalog and inventory. Multiple authorized staff members can operate the platform, subject to their assigned permissions.

1.5 Key Stakeholders

- **Store Owner:** Oversees business operations and administrative access.
- **Administrator:** Manages authorized store operations.
- **Staff Member:** Performs assigned operational tasks.
- **Customer:** Browses products and places orders.
- **Development Team:** Implements and maintains the application.

- **UI/UX Designer:** Designs the customer and administrative interfaces.

2. Product Goals and Success Metrics

2.1 Primary Goals

1 Enable customers to place orders through a simple online shopping experience.
2 Centralize product, category, and inventory management.
3 Provide a reliable order-processing workflow from submission to delivery or cancellation.
4 Allow staff to manage store operations according to role-based permissions.
5 Maintain consistent order, inventory, and customer data.
6 Provide sufficient operational visibility through the Admin Dashboard.

2.2 Success Metrics

The following are proposed metrics. Numerical targets must be agreed upon after establishing a baseline.

Metric	Definition
Order Completion Rate	Delivered orders divided by all submitted orders, measured over a defined period
Checkout Success Rate	Successfully created orders divided by valid checkout submission attempts
Order Processing Time	Average time between order creation and confirmation
Inventory Accuracy	Agreement between recorded stock and physically available stock
Order Cancellation Rate	Cancelled orders divided by all submitted orders
Checkout Error Rate	Failed checkout attempts divided by total checkout attempts
Administrative Task Time	Time required to complete common product and order management tasks

The platform should also be evaluated through usability testing, security testing, and the successful completion of critical end-to-end workflows.

3. User Roles and Permissions

3.1 Customer

A customer can:

- Browse active products and categories.
- Search, sort, and filter products using supported criteria.
- Add available products to the cart.
- Submit an order with the required contact and delivery information.
- Select Cash on Delivery as the payment method.
- View their own order history and order details.
- View and update permitted account information.

3.2 Store Owner / Super Admin

The Store Owner or Super Admin can:

- Access all administrative modules.

- Create and manage staff accounts.
- Assign and revoke administrative permissions.
- Manage products, categories, orders, inventory, and customers.
- View administrative audit records.
- Access store settings available in the approved scope.

The Super Admin role is proposed to ensure that at least one authorized account retains full administrative control.

3.3 Product Manager

Proposed permissions:

- Create and edit products.
- Manage product images, categories, prices, and visibility.
- Update inventory quantities if that permission is granted.
- View stock information.

3.4 Order Manager

Proposed permissions:

- View orders and order details.
- Update order statuses according to the permitted workflow.
- View necessary customer delivery information.
- Record operational notes where supported.

3.5 Customer Support Staff

Proposed permissions:

- View customer profiles and order histories.
- View order details.
- Perform explicitly authorized customer-support actions.
- No access to staff administration or unrelated sensitive data unless separately authorized.

3.6 Permission Model

The platform shall use role-based access control (RBAC).

Permissions must be checked by the backend for every protected operation. Hiding a button in the frontend is not sufficient to prevent unauthorized access.

The initial permission matrix must be approved before implementation.

4. Functional Requirements

FR-01: Authentication and Authorization

- FR-01.1: The system shall allow authorized administrative users to log in securely.
- FR-01.2: The system shall support customer registration and login if customer accounts are enabled.
- FR-01.3: The system shall restrict administrative routes and operations to authorized users.
- FR-01.4: The system shall support logout and session invalidation.
- FR-01.5: The system shall enforce permissions at the backend.
- FR-01.6: The system shall prevent staff from granting themselves higher privileges.

- FR-01.7: The system shall allow the Super Admin to create, deactivate, and assign roles to staff accounts.
- FR-01.8: The system shall protect against brute-force login attempts through appropriate rate limiting.
- FR-01.9: Password recovery and email verification requirements shall be finalized before implementation.

FR-02: Product Management

- FR-02.1: Authorized staff shall be able to create products.
- FR-02.2: Authorized staff shall be able to edit existing products.
- FR-02.3: Authorized staff shall be able to archive products or deactivate their visibility.
- FR-02.4: Permanent deletion shall be restricted when a product is referenced by historical orders, unless a safe archival or retention policy is defined.
- FR-02.5: Each product shall support a name, description, price, category association, stock quantity, and active status.
- FR-02.6: Products shall support one or more images.
- FR-02.7: The system shall validate required fields and reject invalid prices and negative inventory quantities.
- FR-02.8: The system shall support product search, sorting, filtering, and pagination.
- FR-02.9: Customers shall only see products that are active and eligible for display.
- FR-02.10: Product price and stock availability shall be validated again when an order is submitted.

Optional product fields requiring confirmation: SKU, barcode, discount price, product variants such as size or color, tags, brand, weight, and SEO metadata.

FR-03: Category Management

- FR-03.1: Authorized staff shall be able to create, edit, and deactivate categories.
- FR-03.2: Each category shall have a name and may have a description and image.
- FR-03.3: Products shall be associated with one or more categories according to the approved catalog model.
- FR-03.4: The storefront shall display active categories.
- FR-03.5: The system shall prevent deletion or deactivation actions from leaving product associations inconsistent.
- FR-03.6: Category hierarchy shall be limited to the approved design.

Proposed MVP decision: Use a single-level category structure. Nested categories can be added later if required.

FR-04: Product Discovery and Storefront

- FR-04.1: Customers shall be able to browse active products.
- FR-04.2: Customers shall be able to view product details, images, descriptions, prices, and availability.
- FR-04.3: Customers shall be able to browse products by category.
- FR-04.4: Customers shall be able to search products by name.
- FR-04.5: Customers shall be able to sort products by supported criteria.
- FR-04.6: The system shall display clear feedback when no products match a search or category.
- FR-04.7: The system shall prevent customers from adding unavailable products to a valid order.

FR-05: Shopping Cart

- FR-05.1: Customers shall be able to add products to the cart.

- FR-05.2: Customers shall be able to change product quantities and remove items.
- FR-05.3: The system shall validate quantities against current product availability.
- FR-05.4: The system shall calculate item subtotals and the order subtotal.
- FR-05.5: The system shall recalculate totals when quantities change.
- FR-05.6: The system shall validate current prices and stock before creating an order.
- FR-05.7: The cart shall display clear errors if an item becomes unavailable or its price changes.

Proposed MVP behavior: A cart may be maintained locally for guest users or persisted on the server for authenticated users. The final implementation must define cart persistence and guest-to-account cart merging.

FR-06: Checkout and Cash on Delivery

- FR-06.1: Customers shall be able to submit orders using Cash on Delivery.
- FR-06.2: Checkout shall collect the customer's name, phone number, and delivery address.
- FR-06.3: The customer shall review order items, quantities, item prices, subtotal, delivery fee if applicable, and total before submission.
- FR-06.4: The system shall validate all required checkout fields.
- FR-06.5: The system shall verify product availability and current prices on the backend.
- FR-06.6: The system shall create an order with a unique identifier.
- FR-06.7: The system shall record COD as the payment method and the initial payment status as unpaid.
- FR-06.8: The system shall return an order confirmation containing the order reference and summary.
- FR-06.9: The system shall prevent duplicate orders caused by repeated submissions or retries.
- FR-06.10: The system shall reduce or reserve inventory according to the approved stock policy.

Proposed default: Reserve or decrement stock atomically when the order is successfully created, then restore it if the order is cancelled under the approved cancellation rules.

FR-07: Order Management

- FR-07.1: Authorized staff shall be able to view all orders.
- FR-07.2: Staff shall be able to search orders by order number and filter by status and creation date.
- FR-07.3: Staff shall be able to view order items, quantities, price snapshots, totals, customer details, and delivery address.
- FR-07.4: Authorized staff shall be able to update order status through permitted transitions.
- FR-07.5: The system shall record the previous status, new status, actor, and timestamp for each status change.
- FR-07.6: The system shall prevent invalid status transitions.
- FR-07.7: The system shall preserve historical order details when a product is later edited or archived.
- FR-07.8: The system shall support cancellation according to the approved cancellation policy.
- FR-07.9: The system shall provide clear feedback when an order cannot be updated.
- FR-07.10: The system shall distinguish order status from payment status.

FR-08: Customer Account Management

- FR-08.1: Authorized staff shall be able to view a paginated list of customer accounts.
- FR-08.2: Staff shall be able to search customers by name, email, or phone number where available.
- FR-08.3: Staff shall be able to view a customer's profile and order history.
- FR-08.4: Staff shall only be able to edit customer information when explicitly permitted.
- FR-08.5: Authorized staff shall be able to deactivate an account if account suspension is supported.
- FR-08.6: The system shall prevent staff from viewing data beyond their permissions.

- FR-08.7: The system shall preserve order records associated with a customer when the customer account is deactivated.

FR-09: Inventory Management

- FR-09.1: Authorized staff shall be able to view stock quantities.
- FR-09.2: Authorized staff shall be able to update stock quantities with an appropriate permission.
- FR-09.3: The system shall prevent stock from becoming negative through concurrent order submissions.
- FR-09.4: The system shall record inventory changes and their reasons if inventory movement tracking is enabled.
- FR-09.5: The dashboard may highlight products below a configurable stock threshold.

Proposed MVP: Track current stock quantity and prevent overselling. A full inventory ledger, purchase orders, and supplier management are outside the initial scope unless explicitly approved.

FR-10: Admin Dashboard and Reporting

- FR-10.1: The dashboard shall display total orders, recent orders, and current product and customer counts.
- FR-10.2: The dashboard shall display order counts by status.
- FR-10.3: Revenue-related metrics shall clearly define how COD orders, cancellations, and delivered orders are counted.
- FR-10.4: The dashboard shall display recent operational activity where permitted.
- FR-10.5: Dashboard data shall respect the viewing user's permissions.
- FR-10.6: Dashboard figures shall be calculated from consistent and authoritative records.

FR-11: Staff and Role Management

- FR-11.1: The Super Admin shall be able to create staff accounts.
- FR-11.2: The Super Admin shall be able to assign and update roles.
- FR-11.3: The Super Admin shall be able to deactivate staff accounts.
- FR-11.4: The system shall enforce permissions consistently across the interface and API.
- FR-11.5: The system shall prevent unauthorized privilege escalation.
- FR-11.6: The system shall preserve audit records when a staff account is deactivated.
- FR-11.7: The system should prevent accidental removal or deactivation of the last active Super Admin.

FR-12: Administrative Audit Trail

- FR-12.1: The system shall record significant administrative actions, including order status changes, product updates, inventory adjustments, and role changes.
- FR-12.2: Each audit record shall identify the actor, action, target entity, timestamp, and relevant change details.
- FR-12.3: Audit records shall not be editable by ordinary staff.
- FR-12.4: Access to audit records shall be restricted to authorized roles.

FR-13: Customer Notifications

Customer notifications are subject to scope approval.

Proposed MVP: Display order confirmation and current status in the customer interface.

Optional: Email or SMS notifications for order confirmation, shipping, delivery, or cancellation. These require an external provider and additional configuration.

5. User Stories

5.1 Customer Stories

US-C01: Browse Products

As a customer, I want to browse available products so that I can discover items I may want to purchase.

Acceptance criteria:

- Active products are displayed.
- Inactive products are not displayed in normal storefront listings.
- Each product card shows its name, price, and primary image when available.
- An empty catalog displays an appropriate message.

US-C02: View Product Details

As a customer, I want to view a product's details so that I can make an informed purchase decision.

Acceptance criteria:

- The page displays the product name, description, price, images, and availability.
- Unavailable products cannot be added to a valid order.
- A product that does not exist or is not visible returns an appropriate not-found response.

US-C03: Manage Cart

As a customer, I want to add, remove, and update cart items so that I can prepare my order.

Acceptance criteria:

- Quantities must be positive integers.
- Subtotals and totals update when quantities change.
- Current stock is checked before checkout.
- Invalid quantities produce a clear error.

US-C04: Place a COD Order

As a customer, I want to place an order and pay upon delivery so that I can purchase without online payment.

Acceptance criteria:

- Required contact and delivery fields are validated.
- The customer can review the order before submission.
- A successful submission returns an order reference.
- The payment method is recorded as COD and the initial payment status as unpaid.
- A failed submission does not create a partial or duplicate order.

US-C05: Track an Order

As a customer, I want to view my order status so that I know the progress of my purchase.

Acceptance criteria:

- An authenticated customer can view only their own orders.
- Order details include the order reference, items, total, creation date, and current status.
- Unauthorized access to another customer's order is denied.

US-C06: Manage My Account

As a customer, I want to view and update permitted account information so that my contact details remain current.

Acceptance criteria:

- Customers can only access their own account.
- Only approved fields can be edited.
- Invalid data is rejected with understandable feedback.

5.2 Admin and Staff Stories

US-A01: Manage Products

As an authorized product manager, I want to create, update, and archive products so that the storefront stays accurate.

Acceptance criteria:

- Authorized actions succeed with valid data.
- Required fields and prices are validated.
- Unauthorized users cannot perform product mutations.
- Existing order history remains accurate after a product changes.

US-A02: Manage Categories

As an authorized administrator, I want to organize products into categories so that customers can navigate the catalog easily.

Acceptance criteria:

- Authorized users can create and update categories.
- Category changes are reflected in the storefront.
- Product-category associations remain valid.

US-A03: Process Orders

As an order manager, I want to view and update order statuses so that orders move through fulfillment.

Acceptance criteria:

- The order list can be filtered by status.
- Order details display the necessary operational information.
- Invalid status transitions are rejected.
- Every successful status change is recorded in the audit trail.

US-A04: Review Customer Accounts

As authorized support staff, I want to review customer details and order history so that I can help customers with their purchases.

Acceptance criteria:

- Customer search returns matching records.
- Customer order history is visible to authorized staff.
- Sensitive data is protected by permissions.

US-A05: Manage Staff Permissions

As the Super Admin, I want to assign staff roles so that team members can perform only their assigned duties.

Acceptance criteria:

- The Super Admin can create staff accounts and assign roles.
- Unauthorized staff cannot modify roles or grant privileges.
- Deactivated staff cannot access protected administrative functions.

US-A06: Monitor Store Operations

As the Store Owner, I want to see recent orders and key store statistics so that I can monitor daily activity.

Acceptance criteria:

- Counts and status summaries are available.
- The displayed data follows documented definitions.
- Unauthorized users cannot access restricted metrics.

6. Detailed Workflows

6.1 Customer Purchase Workflow

1 The customer opens the storefront.
2 The customer browses or searches active products.
3 The customer opens a product detail page.
4 The customer selects a quantity and adds the product to the cart.
5 The customer reviews cart items and quantities.
6 The customer proceeds to checkout.
7 The customer provides the required contact and delivery details.
8 The customer reviews the final order summary.
9 The customer selects Cash on Delivery.
10 The backend validates product status, stock, prices, and checkout fields.
11 The system creates the order and its item snapshots atomically.
12 The system applies the approved stock reservation or decrement policy.
13 The system records the order with its initial status and COD payment status.
14 The customer receives an order reference and confirmation.
15 The order appears in the Admin Dashboard.

Failure handling:

- If stock is insufficient, the order is not created and the customer is asked to adjust the cart.
- If the price changed, the customer is shown the updated total and asked to confirm again.
- If the request is repeated, an idempotency mechanism prevents duplicate order creation.
- If the database transaction fails, partial order and stock updates are rolled back.

6.2 Product Creation Workflow

1 An authorized staff member opens Products.
2 The staff member selects Add Product.
3 The staff member enters product details.
4 The staff member selects a category and uploads images.
5 The staff member enters the price and stock quantity.
6 The staff member selects the visibility status.
7 The system validates the form and uploaded files.
8 The system saves the product.
9 The product appears in the product list.
10 If active and otherwise eligible for display, the product becomes available on the storefront.

Failure handling: Invalid fields, unauthorized operations, and upload errors must produce actionable messages without saving inconsistent data.

6.3 Order Processing Workflow

Proposed standard flow:

`Pending → Confirmed → Processing → Shipped → Delivered`

Cancellation may be allowed from selected earlier statuses according to the approved business policy.

1. A new order is created with status Pending.
2. Authorized staff review the order and delivery details.
3. Staff confirm the order or cancel it if permitted.
4. Confirmed orders proceed to Processing.
5. Once dispatched, the status changes to Shipped.
6. Once delivered, the status changes to Delivered.
7. Every successful transition records the actor, timestamp, and previous and new statuses.
8. The customer can view the latest status.

COD payment handling:

- The order starts with payment status Unpaid.
- Delivery does not automatically prove payment collection.
- Authorized staff or an approved delivery integration must record collection before the payment status becomes Paid.
- Any payment discrepancy must follow the store's operational policy.

Optional: Add a `Delivery Failed` status if the business needs delivery-attempt tracking.

6.4 Cancellation and Stock Restoration Workflow

1. An authorized user requests cancellation.
2. The system checks the current order status and the user's permission.
3. The system verifies that cancellation is allowed.
4. The system updates the order status and records the reason where required.
5. The system restores reserved or deducted stock according to the inventory policy.
6. The system records the changes atomically and adds an audit record.
7. The customer can view the updated status.

The system must prevent stock restoration from being applied more than once for the same cancellation.

6.5 Staff Administration Workflow

1. The Super Admin opens Staff Management.
2. The Super Admin creates a staff account.
3. The Super Admin assigns a predefined role.
4. The system validates the requested permissions.
5. The staff account is created and the action is audited.
6. The staff member signs in and receives only the permissions associated with the assigned role.
7. If the account is deactivated, future access is denied and active sessions are revoked according to the session policy.

7. Business Rules

7.1 Product Rules

- Only active products can be purchased.
- A product price must be greater than or equal to zero unless the business requires strictly positive prices.

- Stock quantities must be non-negative integers.
- Archived products must remain available in historical order records.
- The product catalog must not be allowed to corrupt historical order details.
- Variant-level inventory is required only if product variants are approved.

7.2 Pricing Rules

- Product prices must be stored using a decimal-safe representation or integer minor units, not binary floating-point arithmetic.
- Order items must store a snapshot of the unit price at order creation.
- The order total must be calculated on the server.
- The customer must be informed of applicable delivery fees before confirming the order.
- Currency must be explicitly configured. EGP is a possible default for an Egypt-focused store, but this requires approval.

7.3 Inventory Rules

- The server is the authoritative source for available stock.
- Concurrent purchases must not result in negative stock.
- Stock changes and order creation must be transactionally consistent.
- Cancellation must restore stock only when the approved stock policy requires it.
- The system must prevent duplicate stock deductions or restorations.

7.4 Order Rules

- Each order must have a unique, non-guessable public reference or an appropriately protected identifier.
- Every order must contain at least one item.
- An order must retain item names, unit prices, quantities, and relevant product details as snapshots.
- Only permitted status transitions may be applied.
- Customers may access only their own orders.
- Staff can perform only the order operations allowed by their permissions.
- Cancellation rules must define which statuses can be cancelled and by whom.

7.5 Cash on Delivery Rules

- COD is the only supported payment method in the initial scope.
- A COD order starts as Unpaid.
- Payment collection must be recorded separately from order delivery status.
- Online payment processing and payment gateway integrations are not required for the MVP.
- Refunds, partial collections, and delivery-company cash reconciliation are outside the initial scope unless approved.

7.6 Customer Rules

- Customer account information must be protected.
- Account deactivation must not silently erase historical orders.
- Duplicate account handling must use an approved identity policy, such as unique verified email or phone where applicable.
- Guest checkout is not confirmed and must be explicitly decided.

7.7 Administrative Rules

- All protected operations require server-side authorization.
- Privilege changes must be restricted to authorized roles.
- Significant administrative changes must be auditable.
- The system should prevent deactivation of the last active Super Admin.
- Staff must not be able to access passwords or retrieve password hashes through application APIs.

8. Order Status and Payment Status Models

8.1 Order Status

Proposed MVP statuses:

Status	Meaning
Pending	Order created and awaiting review
Confirmed	Order accepted by the store
Processing	Order is being prepared
Shipped	Order has been dispatched
Delivered	Order has been delivered
Cancelled	Order has been cancelled under the approved policy

The allowed transitions must be enforced by the backend. Reopening cancelled orders or changing Delivered orders back to an earlier state requires an explicit policy.

8.2 Payment Status

Proposed statuses:

Status	Meaning
Unpaid	COD payment has not been collected
Paid	Payment has been collected and recorded
Failed	A payment attempt failed, if such a concept is later needed
Refunded	Payment was refunded, if refunds are supported

For the COD-only MVP, Unpaid and Paid are sufficient. Failed and Refunded are reserved for future payment requirements unless the business defines another need.

9. Data Entities and Relationships

The following is a proposed logical data model, not a finalized database schema.

9.1 User

Represents a customer or administrative user.

Fields:

- `id`
- `full_name`
- `email` (nullable if the approved account model permits)
- `phone`
- `password_hash` for password-based accounts

- `user_type` or associated role
- `account_status`
- `created_at`
- `updated_at`

Sensitive authentication fields must never be returned by public APIs.

9.2 Role

Fields:

- `id`
- `name`
- `description`
- `created_at`

9.3 Permission

Fields:

- `id`
- `key`
- `description`

9.4 RolePermission

Associates roles with permissions.

Fields:

- `role_id`
- `permission_id`

9.5 UserRole

Associates users with roles where the approved model permits one or more roles.

Fields:

- `user_id`
- `role_id`

A simpler implementation may store one role per staff account. This decision must be made before schema finalization.

9.6 CustomerAddress

Fields:

- `id`
- `customer_id`
- `recipient_name`
- `phone`
- `address_line`
- `city`
- `region`
- `postal_code` (optional)
- `delivery_notes` (optional)
- `created_at`
- `updated_at`

9.7 Category

Fields:

- `id`
- `name`
- `slug`
- `description` (optional)
- `image_url` (optional)
- `status`
- `created_at`
- `updated_at`

9.8 Product

Fields:

- `id`
- `name`
- `slug`
- `description`
- `price`
- `stock_quantity`
- `category_id` if each product belongs to one category
- `status`
- `created_at`
- `updated_at`
- `archived_at` (optional)

If products can belong to multiple categories, use a junction table such as `ProductCategory` instead of a single `category_id`.

9.9 ProductImage

Fields:

- `id`
- `product_id`
- `image_url`
- `alt_text` (optional)
- `sort_order`
- `created_at`

9.10 Cart and CartItem

Cart

- `id`
- `customer_id` or guest/session identifier
- `created_at`
- `updated_at`

CartItem

- `id`
- `cart_id`

- `product_id`
- `quantity`
- `created_at`
- `updated_at`

Guest cart persistence and expiration are implementation decisions to confirm.

9.11 Order

Fields:

- `id`
- `order_number`
- `customer_id` (nullable only if guest checkout is approved)
- `recipient_name`
- `recipient_phone`
- `shipping_address_snapshot`
- `subtotal`
- `delivery_fee`
- `total_amount`
- `currency`
- `payment_method`
- `payment_status`
- `order_status`
- `customer_notes` (optional)
- `created_at`
- `updated_at`
- `cancelled_at` (optional)

The order must retain a snapshot of the delivery information so later address edits do not change historical orders.

9.12 OrderItem

Fields:

- `id`
- `order_id`
- `product_id` (nullable if product deletion policy requires it)
- `product_name_snapshot`
- `unit_price_snapshot`
- `quantity`
- `line_total`

The line total is calculated from the stored unit price and quantity using the agreed pricing rules.

9.13 OrderStatusHistory

Fields:

- `id`
- `order_id`
- `previous_status`
- `new_status`
- `changed_by_user_id`
- `reason` (optional)

- `created_at`

9.14 InventoryMovement (Optional for MVP)

Fields:

- `id`
- `product_id`
- `quantity_change`
- `reason`
- `reference_type`
- `reference_id`
- `created_by_user_id`
- `created_at`

This entity is recommended if detailed stock auditing is needed. It can be deferred if the MVP only tracks the current stock quantity.

9.15 AuditLog

Fields:

- `id`
- `actor_user_id`
- `action`
- `entity_type`
- `entity_id`
- `before_data` (appropriately filtered)
- `after_data` (appropriately filtered)
- `created_at`

Sensitive credentials, session tokens, and unnecessary personal data must not be included in audit payloads.

9.16 Core Relationships

- A user may own multiple orders.
- An order contains one or more order items.
- A product can appear in multiple historical order items.
- A category can contain multiple products.
- A user may have one or more roles depending on the approved authorization model.
- A role may contain multiple permissions.
- An order may have multiple status-history records.
- A product may have multiple images.
- A cart contains multiple cart items.

10. API Requirements

10.1 API Design Principles

- Use RESTful HTTP endpoints returning JSON.
- Use consistent request and response formats.
- Validate inputs on the server.
- Enforce authentication and authorization on protected endpoints.

- Use pagination for list endpoints.
- Use consistent error responses and appropriate HTTP status codes.
- Use database transactions for order creation and inventory changes.
- Use idempotency protection for order creation where retries may occur.
- Version the API, for example under `/api/v1`.
- Never trust frontend-provided totals, prices, roles, or stock availability.

10.2 Authentication Endpoints

Method	Endpoint	Purpose
POST	`/api/v1/auth/login`	Authenticate a user
POST	`/api/v1/auth/logout`	End the current session
GET	`/api/v1/auth/me`	Retrieve the authenticated user's profile and permissions
POST	`/api/v1/auth/register`	Register a customer, if registration is enabled
POST	`/api/v1/auth/forgot-password`	Request password recovery, if supported
POST	`/api/v1/auth/reset-password`	Complete password recovery, if supported

10.3 Public Product and Category Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/products`	List active products
GET	`/api/v1/products/{id-or-slug}`	Retrieve product details
GET	`/api/v1/categories`	List active categories
GET	`/api/v1/categories/{id-or-slug}/products`	List products in a category

Supported product-list query parameters may include `search`, `category`, `sort`, `page`, and `limit`.

10.4 Cart and Checkout Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/cart`	Retrieve the current user's cart
POST	`/api/v1/cart/items`	Add an item to the cart
PATCH	`/api/v1/cart/items/{id}`	Update item quantity
DELETE	`/api/v1/cart/items/{id}`	Remove an item
POST	`/api/v1/orders`	Create an order
GET	`/api/v1/orders/{orderNumber}`	Retrieve an order only after proper authorization

Guest cart and checkout endpoints must be designed separately if guest checkout is approved.

10.5 Customer Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/customers/me`	Retrieve the current customer's profile
PATCH	`/api/v1/customers/me`	Update permitted profile fields
GET	`/api/v1/customers/me/orders`	List the customer's own orders

Method	Endpoint	Purpose
GET	`/api/v1/customers/me/orders/{orderNumber}`	Retrieve one of the customer's own orders

10.6 Admin Product and Category Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/admin/products`	List products, including inactive products
POST	`/api/v1/admin/products`	Create a product
GET	`/api/v1/admin/products/{id}`	Retrieve an administrative product record
PATCH	`/api/v1/admin/products/{id}`	Update a product
DELETE	`/api/v1/admin/products/{id}`	Archive or safely delete a product according to policy
GET	`/api/v1/admin/categories`	List categories
POST	`/api/v1/admin/categories`	Create a category
PATCH	`/api/v1/admin/categories/{id}`	Update a category
DELETE	`/api/v1/admin/categories/{id}`	Deactivate or delete a category according to policy

10.7 Admin Order and Customer Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/admin/orders`	List and filter orders
GET	`/api/v1/admin/orders/{id}`	Retrieve order details
PATCH	`/api/v1/admin/orders/{id}/status`	Update an order status
GET	`/api/v1/admin/customers`	List customer accounts
GET	`/api/v1/admin/customers/{id}`	Retrieve customer details
GET	`/api/v1/admin/customers/{id}/orders`	Retrieve a customer's order history
PATCH	`/api/v1/admin/customers/{id}/status`	Update customer account status if permitted

10.8 Staff, Roles, and Dashboard Endpoints

Method	Endpoint	Purpose
GET	`/api/v1/admin/dashboard`	Retrieve authorized dashboard statistics
GET	`/api/v1/admin/staff`	List staff accounts
POST	`/api/v1/admin/staff`	Create a staff account
PATCH	`/api/v1/admin/staff/{id}`	Update allowed staff details
PATCH	`/api/v1/admin/staff/{id}/status`	Activate or deactivate staff
GET	`/api/v1/admin/roles`	List roles and permissions
POST	`/api/v1/admin/staff/{id}/roles`	Assign roles to staff
GET	`/api/v1/admin/audit-logs`	Retrieve audit logs for authorized users

The exact endpoint list must be aligned with the final database model, authentication mechanism, and permissions matrix.

10.9 Example Order Creation Request

```
{
  "items": [
    {
      "product_id": "prod_123",
      "quantity": 2
    }
  ],
  "payment_method": "COD",
  "shipping_address": {
    "recipient_name": "Customer Name",
    "phone": "01000000000",
    "address_line": "Street and building details",
    "city": "Suez",
    "region": "Suez"
  }
}
```

The request intentionally excludes product prices and totals. The server must retrieve current prices, validate stock, calculate the total, and create the order.

10.10 Example Order Status Update Request

```
{
  "status": "CONFIRMED",
  "reason": "Order reviewed and confirmed"
}
```

The backend must validate the user's permission, current order status, and permitted transition before saving the change.

10.11 Standard Error Format

Proposed format:

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "One or more products are no longer available in the requested quantity.",
    "details": []
  }
}
```

Expected HTTP status codes include:

- `200 OK`: Successful retrieval or update.
- `201 Created`: Successful resource creation.
- `400 Bad Request`: Invalid request structure.
- `401 Unauthorized`: Missing or invalid authentication.
- `403 Forbidden`: Authenticated user lacks permission.
- `404 Not Found`: Resource not found or intentionally hidden.
- `409 Conflict`: Stock conflict, invalid state transition, or duplicate conflict.
- `422 Unprocessable Content`: Field validation failure, if this convention is adopted.
- `429 Too Many Requests`: Rate limit exceeded.
- `500 Internal Server Error`: Unexpected server error.

11. Non-Functional Requirements

11.1 Security

- Passwords must be stored using an appropriate password-hashing algorithm.
- All production traffic must use HTTPS.
- Protected API operations must enforce authentication and role-based authorization.
- Input validation and output encoding must mitigate common injection and cross-site scripting risks.
- State-changing requests must be protected against applicable CSRF risks when cookie-based authentication is used.
- Administrative login and password recovery must use secure session and token handling.
- The system must rate-limit sensitive endpoints.
- Secrets and credentials must not be hardcoded into source code or exposed in frontend bundles.
- Administrative actions must be auditable.
- Personal data must be collected and retained only as necessary for the approved business purpose.

11.2 Performance

Proposed targets, subject to infrastructure and load testing:

- Common storefront pages should render within 2.5 seconds at the 75th percentile under the agreed test conditions.
- Typical catalog and administrative list API requests should complete within 500 ms at the 95th percentile under the agreed test conditions, excluding large uploads and external dependencies.
- Product listings and order tables must support pagination.
- Images must be appropriately resized and optimized.

11.3 Availability and Reliability

- Database transactions must preserve consistency during order creation and cancellation.
- Errors must be logged without exposing sensitive information.
- Backups and restoration procedures must be defined before production launch.
- Critical operations must provide clear success or failure responses.
- The application must avoid duplicate order creation during retries.

11.4 Usability and Accessibility

- The interface must provide clear loading, empty, success, and error states.
- Destructive actions should require confirmation where appropriate.
- Forms must identify required fields and explain validation errors.
- The interface should support keyboard navigation and accessible labels.
- The customer storefront and Admin Dashboard must be responsive across supported screen sizes.
- The language, RTL support, and accessibility target must be confirmed before design approval.

11.5 Maintainability

- Frontend, backend, and data-access responsibilities should be separated.
- Business rules must be enforced in a central backend layer.
- API contracts must be documented.
- Automated tests should cover critical workflows and business rules.
- Production and development configurations must be separated.

12. Acceptance Criteria for MVP Release

The MVP is considered functionally ready when all required criteria below pass.

AC-01: Administrative Access

- An authorized Admin can log in and access permitted pages.
- A customer cannot access protected administrative operations.
- A staff member cannot execute operations outside their assigned permissions.

AC-02: Product Management

- An authorized user can create, edit, and archive a product.
- Invalid prices, missing required fields, and negative quantities are rejected.
- Archived products are excluded from normal storefront listings.
- Historical order item snapshots remain unchanged after product edits.

AC-03: Categories

- Authorized staff can create and edit categories.
- Customers can browse active categories.
- Category changes do not create invalid product associations.

AC-04: Storefront

- Customers can browse and view active products.
- Search and category filtering return appropriate results.
- Unavailable or inactive products cannot be purchased.

AC-05: Cart and Checkout

- Customers can add, update, and remove cart items.
- The server calculates the authoritative total.
- Checkout rejects invalid contact details, insufficient stock, and invalid quantities.
- A successful COD checkout creates one valid order with a unique reference.
- Repeated requests do not create duplicate orders.
- The initial payment status is Unpaid.

AC-06: Inventory

- Stock cannot become negative when multiple customers order concurrently.
- Order creation and stock updates are atomic.
- Cancellation does not restore the same stock more than once.

AC-07: Order Processing

- Authorized staff can view orders and update statuses through allowed transitions.
- Invalid transitions are rejected.
- Status history records the actor and timestamp.
- Customers can view only their own order details.

AC-08: Customer Management

- Authorized staff can search customers and view their permitted details.
- Customer account changes respect role permissions.
- Account deactivation does not destroy historical order records.

AC-09: Staff Management

- The Super Admin can create and deactivate staff accounts and assign roles.
- Deactivated staff cannot access protected administrative operations.
- Unauthorized role changes are rejected.

AC-10: Dashboard and Audit

- Dashboard counts are consistent with the underlying records and documented metric definitions.
- Important administrative changes create audit records.
- Audit records cannot be modified by ordinary staff.

AC-11: Error Handling and Security

- API endpoints return documented error responses.
- Unauthorized and invalid requests do not mutate protected data.
- Sensitive credentials are not returned by APIs.
- Critical workflows pass automated or documented end-to-end tests.

13. MVP Scope

In Scope

1. Customer-facing responsive storefront.
2. Product listing and product details.
3. Category browsing.
4. Search and basic product filtering.
5. Shopping cart.
6. Checkout with Cash on Delivery.
7. Customer order history and status tracking.
8. Admin authentication.
9. Role-based access control for Admin and Staff.
10. Product CRUD and product image management.
11. Category management.
12. Order listing, details, status updates, and cancellation rules.
13. Customer account listing and permitted account management.
14. Basic inventory tracking and overselling prevention.
15. Dashboard with essential operational statistics.
16. Administrative audit logging for critical actions.
17. Input validation, authorization, error handling, and automated tests.

Proposed Phase 2

These items are not part of the MVP unless separately approved:

- Online payment gateways.
- Email and SMS notifications.
- Coupons, promotions, and discount campaigns.
- Product reviews and ratings.
- Wishlist functionality.
- Product variants and variant-level inventory.
- Advanced sales reports and CSV exports.
- Delivery company integrations and shipment tracking.

- Supplier management and purchase orders.
- Returns, exchanges, and refund workflows.
- Multi-language storefront.
- Multiple currencies.
- Advanced analytics and marketing integrations.

14. Out of Scope

The following are explicitly excluded from the initial release:

- Multi-vendor marketplace functionality.
- Seller registration, seller dashboards, and seller commissions.
- Online payment processing.
- Subscription or recurring billing.
- Native Android or iOS applications.
- Automated courier booking and external shipment tracking.
- Automated tax or accounting integrations.
- Warehouse management across multiple locations.
- Enterprise resource planning (ERP) integrations.
- AI-powered product recommendations.
- Loyalty points and referral programs.
- Automated marketing campaigns.

Any excluded feature may be considered in a future release after requirements, dependencies, and business value are assessed.

15. Dependencies and Assumptions

Dependencies

- A hosting environment and domain.
- A database and backend API.
- Image storage for product images.
- Authentication and session-management infrastructure.
- A backup and recovery mechanism.
- A method for configuring store settings and delivery fees.

Assumptions Requiring Approval

- The business operates one store and one shared product catalog.
- Multiple administrative users are supported.
- Cash on Delivery is the only payment method in the MVP.
- A single currency is used.
- Stock is managed within the platform.
- The platform has a defined policy for cancellations and stock restoration.
- Product variants are not required in the initial version.
- The platform has a defined customer account and guest-checkout policy.
- Store staff manually update shipping and delivery statuses unless an integration is added.

These assumptions must be confirmed before the development team finalizes the schema and implementation plan.

16. Open Questions

The following decisions remain unresolved and must not be treated as confirmed requirements.

1. **Customer accounts:** Is registration and login mandatory, or should customers be able to order as guests?
2. **Language:** Will the storefront be Arabic, English, or bilingual? Is full RTL support required?
3. **Currency:** Will all prices be in EGP?
4. **Delivery fees:** Is there a fixed delivery fee, a location-based fee, or free delivery above a threshold?
5. **Product categories:** Can a product belong to multiple categories? Are nested categories required?
6. **Product variants:** Do products need sizes, colors, or other variants?
7. **Inventory timing:** Should stock be deducted at order creation, confirmation, or shipment?
8. **Cancellation:** Which statuses can be cancelled, and can customers request cancellation themselves?
9. **Order confirmation:** Should orders be confirmed automatically or manually by staff?
10. **Delivery workflow:** Is manual status updating sufficient, or is courier integration needed?
11. **Staff roles:** Which predefined roles and permissions are required for launch?
12. **Notifications:** Are in-app order updates sufficient, or are email/SMS notifications required?
13. **Dashboard analytics:** Which sales and revenue definitions should be used for COD orders?
14. **Customer management:** Should staff be able to edit customer contact information or only view it?
15. **Storefront design:** Are there existing brand guidelines, colors, logos, or reference websites?

17. Definition of Done

A feature is considered complete when:

- Its approved functional requirements are implemented.
- Its acceptance criteria pass.
- Required validation and authorization are implemented on the backend.
- Relevant error and empty states are handled.
- Automated tests cover critical logic.
- API documentation is updated where applicable.
- The feature has been reviewed for usability, security, and responsiveness.
- No unresolved critical defects remain.

The MVP release is ready when all mandatory acceptance criteria pass in the agreed test environment and the business approves the remaining configuration decisions.

18. Recommended Implementation Sequence

1. Finalize open questions and approve the MVP scope.
2. Define the database schema and permission matrix.
3. Implement authentication and authorization.
4. Implement categories, products, and image management.
5. Implement storefront browsing and product details.

6 Implement cart, checkout, order creation, and COD handling.

7 Implement inventory consistency and order-processing workflows.

8 Implement customer account management.

9 Implement staff management, dashboard statistics, and audit logging.

10 Complete integration testing, security testing, and responsive UI validation.

11 Deploy to a staging environment and perform business acceptance testing.

12 Deploy the approved MVP to production with backups and monitoring configured.

Final Deliverable: A responsive single-store e-commerce web application with a customer storefront, a multi-user Admin Dashboard with role-based permissions, product and category management, customer and order management, inventory controls, and Cash on Delivery checkout.